

Standard Library

IO, files, strings, memory, etc

Recap Input/Output

Printf and format strings

```
int printf(const char *format_string, ...);
```

Parameters:

- Format string
- ... → *variadic arguments*, variable size list of arguments (zero or more)

Returns:

- Number of output characters

Format strings

Between double quotes (string, not character).

Contains

- Regular printable characters, including escaped characters (e.g. `\t \n \r`); these are outputted *as is*
- Conversion specifiers: sequence of characters starting with '%'; these are replaced with the corresponding argument from the ... list

Conversion specifiers (commonly used)

- `%d` ->
- `%c` ->
- `%s` ->

Format strings

Between double quotes (string, not character).

Contains

- Regular printable characters, including escaped characters (e.g. `\t \n \r`); these are outputted *as is*
- Conversion specifiers: sequence of characters starting with '%'; these are replaced with the corresponding argument from the ... list

Conversion specifiers (commonly used)

- `%d` -> prints integer
- `%c` -> prints single character
- `%s` -> prints string

Format strings

The number of arguments in the ... list must match the number of conversion specifiers in the format string.

What happens when we use

- `printf(val); // val is char or char[] variable`

Format strings

The number of arguments in the ... list must match the number of conversion specifiers in the format string.

What happens when we use

- `printf(val);` // `val` is `char[]` variable
 - `val` is treated as a format string
 - `printf` starts at the address of `val` and prints each regular character

```
void main(void){  
    char string[] = "hello World\n";  
    printf(string);  
}
```

```
$ ./printf  
hello world
```

Format strings

The number of arguments in the ... list must match the number of conversion specifiers in the format string.

What happens when we use

- `printf(val); // val is char[] variable`
 - **val** is treated as a format string
 - `printf` starts at the address of `val` and prints each regular character
 - What if **val** contains conversion specifiers (%)?

Format strings

Undefined behaviour!

The number of arguments in the ... list must match the number of conversion specifiers in the format string.

What happens when we use

- `printf(val);` // `val` is `char[]` variable
 - `val` is treated as a format string
 - `printf` starts at the address of `val` and prints each regular character
 - What if `val` contains conversion specifiers (%)?



```
void main(void){  
    char string[] = "hello %d world\n";  
    printf(string);  
}
```

```
$ ./printf  
hello 1981949448 world
```

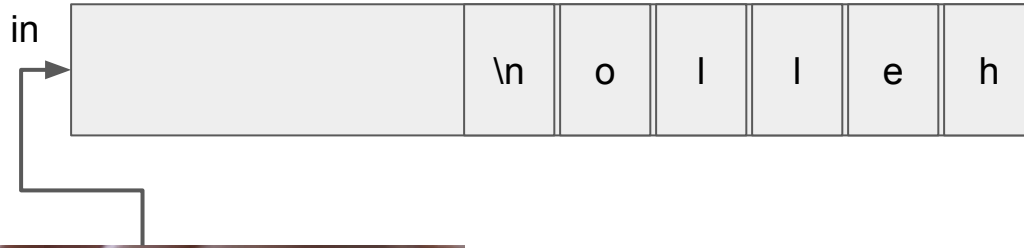
getchar

```
int getchar();
```

- Reads one character from the input buffer and returns its ASCII code

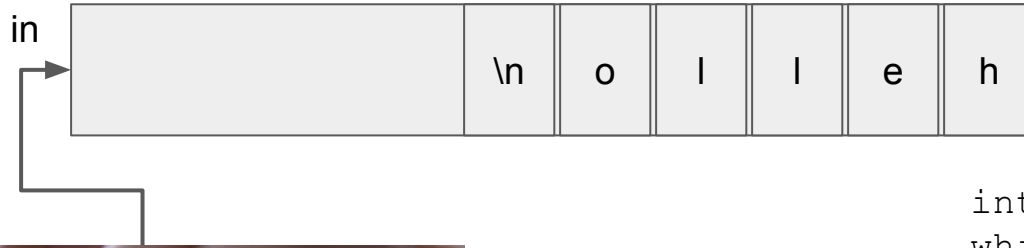
getchar

Input buffer



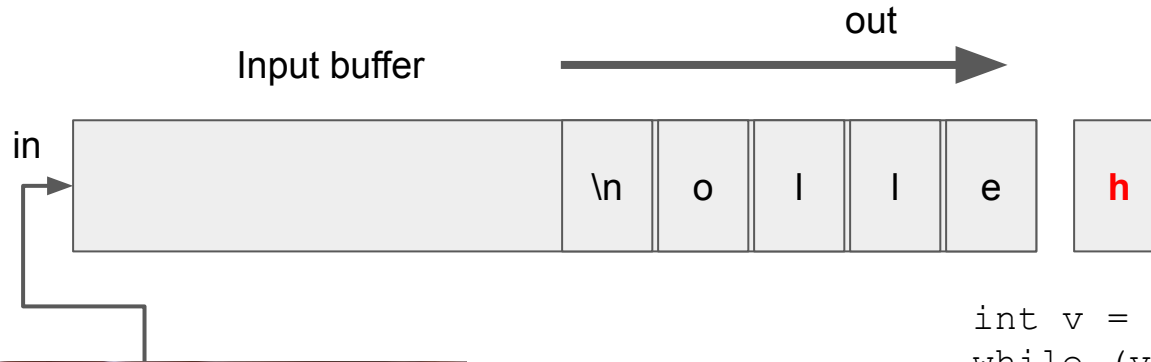
getchar

Input buffer



```
int v = 0;int i=0;
while (v != '\n') {
    v = getchar();
    printf("%d: %c", i, v);
    i++;
}
printf("%c", v);
```

getchar



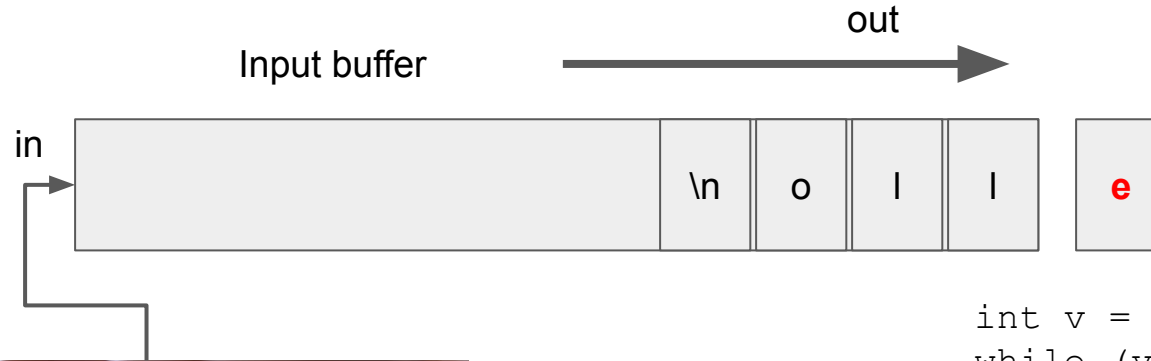
0: h



```
int v = 0; int i = 0;
while (v != '\n') {
    v = getchar();
    printf("%d: %c", i, v);
    i++;
}
printf("%c", v);
```

getchar

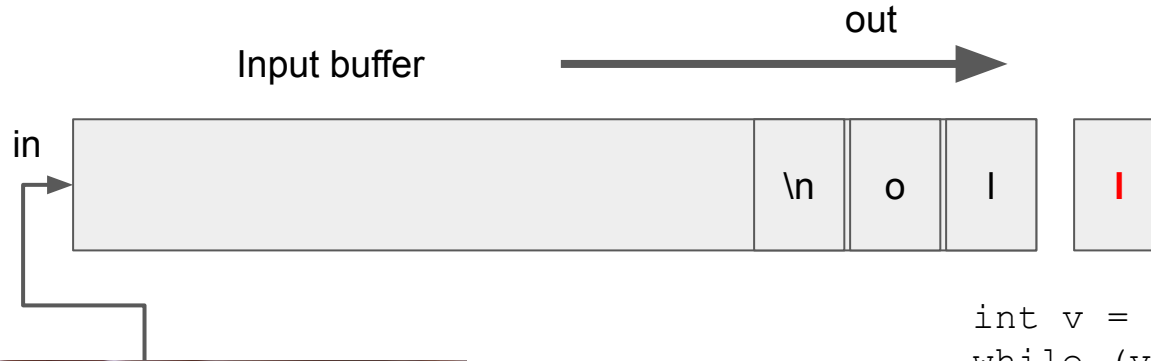
```
0: h  
1: e
```



```
int v = 0; int i = 0;  
while (v != '\n') {  
    v = getchar();  
    printf("%d: %c", i, v);  
    i++;  
}  
printf("%c", v);
```

getchar

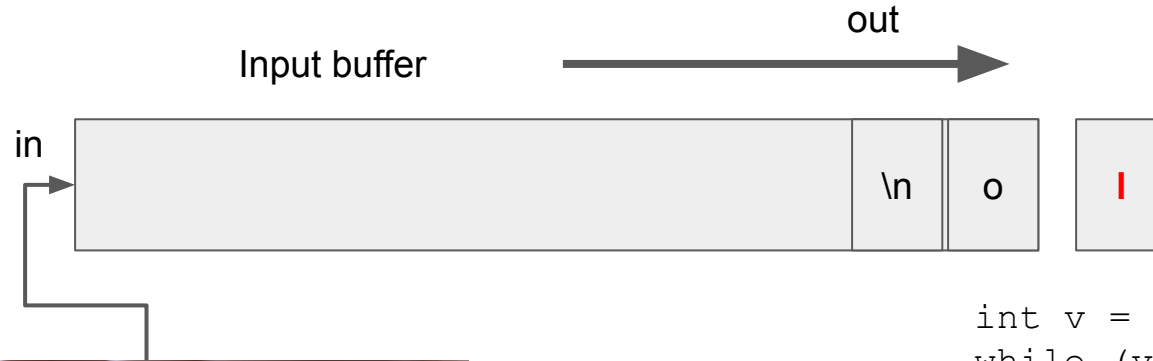
```
0: h  
1: e  
2: l
```



```
int v = 0; int i = 0;  
while (v != '\n') {  
    v = getchar();  
    printf("%d: %c", i, v);  
    i++;  
}  
printf("%c", v);
```

getchar

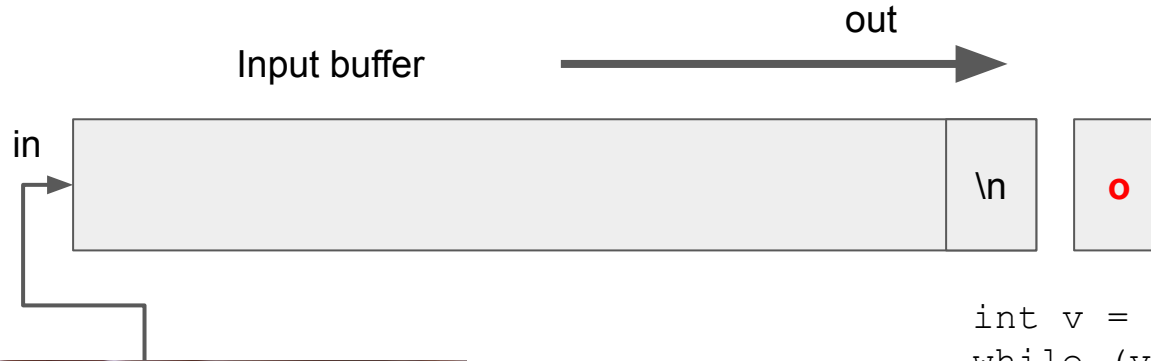
```
0: h
1: e
2: l
3: l
```



```
int v = 0; int i = 0;
while (v != '\n') {
    v = getchar();
    printf("%d: %c", i, v);
    i++;
}
printf("%c", v);
```

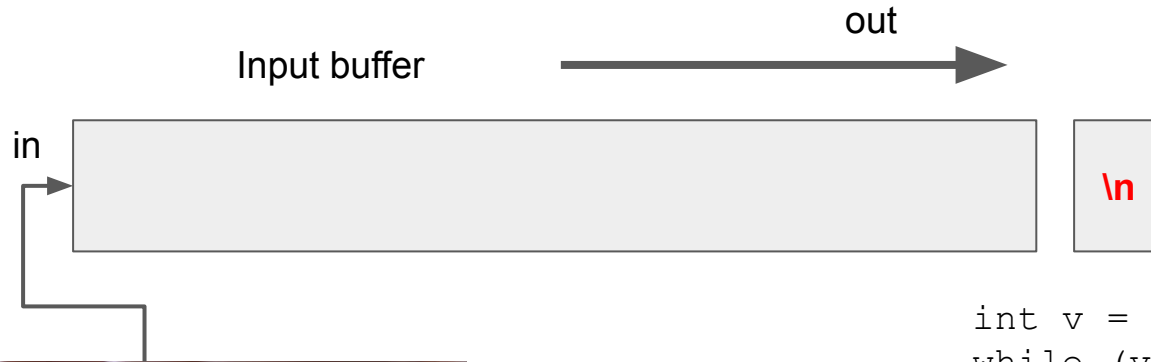

getchar

```
0: h
1: e
2: l
3: l
4: o
```



```
int v = 0; int i = 0;
while (v != '\n') {
    v = getchar();
    printf("%d: %c", i, v);
    i++;
}
printf("%c", v);
```

getchar



```
0: h
1: e
2: l
3: l
4: o
5:
```

```
int v = 0; int i = 0;
while (v != '\n') {
    v = getchar();
    printf("%d: %c", i, v);
    i++;
}
printf("%c", v);
```



scanf

```
int scanf(const char *format_string, args...);
```

What does it return?

What are the arguments?

scanf

```
int scanf(const char *format_string, args...);
```

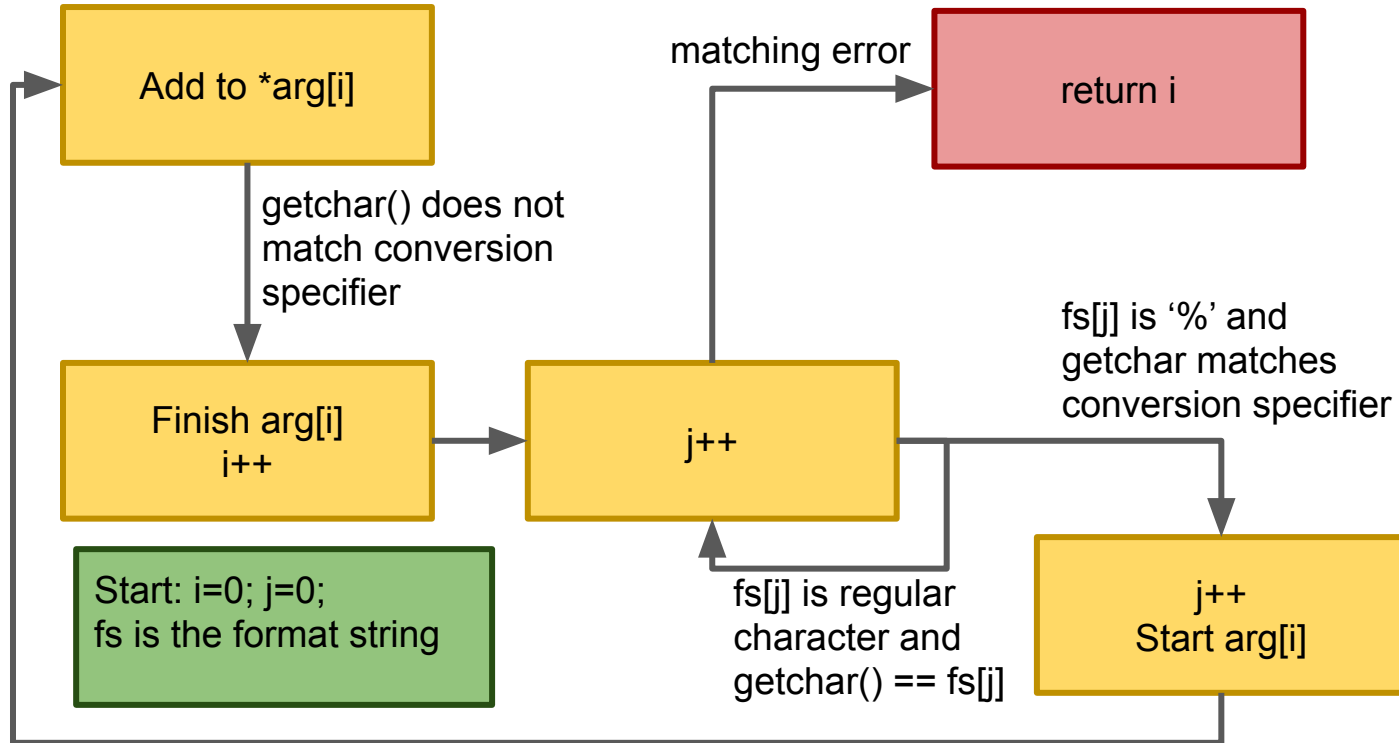
What does it return?

- The number of arguments matched

What are the arguments?

- Pointers
- Specifically, the memory addresses where the user input should be stored.

scanf : processing input



Files

FILE objects

Standard C library (`<stdlib.h>`) provides the FILE abstraction.

Always a pointer: `FILE *f;`

What a FILE object represents

- Underlying file descriptor (filesystem file)
- Position in the file
- File input/output buffer
- File access mode (input, output, or update)
- Type of file
 - Binary: the input is written *as is*
 - Text: only printable characters; some characters will be translated before writing.

Opening files

Similar to Python

Syntax: **FILE** *fopen (const char *filename, const char *mode);

Modes:

“r”: read, file_pos at start

“r+”: read & write, file_pos at start

“w”: write, truncate existing or create file if doesn't exist, file_pos at start

“w+”: read & write, truncate existing or create file if doesn't exist, file_pos at start

“a”: append, create file if doesn't exist, file_pos at end

“a+”: read & append, create file if doesn't exist, read at start, append at end.

Examples

```
#include <stdlib.h>
```

```
...
```

```
FILE *f;
```

```
f = fopen("letter_values", "r");
```

Error handling:

- What if the file isn't there? **fopen** returns **NULL (zero)**

Examples

```
#include <stdlib.h>
```

```
...
```

```
FILE *f;
```

```
f = fopen("letter_values", "r");
```

```
if (f == NULL){ printf("File does not exist\n"); // exit}
```

Error handling:

- What if the file isn't there? **fopen** returns **NULL (zero)**

Examples

```
#include <stdlib.h>
```

```
...
```

```
FILE *f;
```

```
if ((f = fopen("letter_values", "r")) == NULL){  
    printf("File does not exist\n"); // exit  
}
```

Error handling:

- What if the file isn't there? **fopen** returns **NULL (zero)**

Closing files

```
#include <stdlib.h>
```

```
...
```

```
FILE *f;
```

```
if ((f = fopen("letter_values", "r")) == NULL){  
    printf("File does not exist\n"); // exit  
}
```

```
...
```

```
fclose(f);
```

Special files: input, output, error

Every application starts up with three open files:

- **stdin**: standard input
- **stdout**: standard output
- **stderr**: standard error

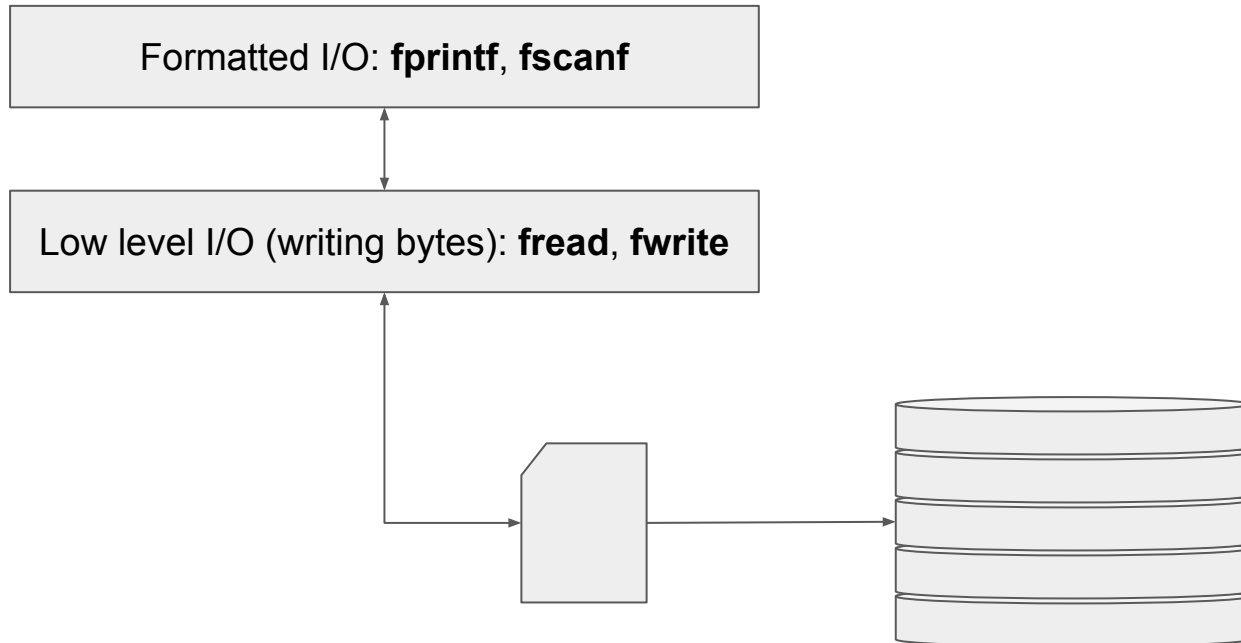
These are bound to the terminal where the application is run.

`stdin`, `stdout`, `stderr` are **extern FILE *** variables declared in the `stdio.h` header

They can be used in the program (writing, reading, reopening, etc).

File input/output

File input/output



Low level: fwrite

Writing objects to files

`size_t fwrite(const void *buffer, size_t size, size_t count, FILE *stream);`

- Data is stored at the address **buffer**
- Data is considered to be an array of **count** elements, each of **size** bytes
- All the **count** elements are written to the **stream** file.

fwrite example with basic types

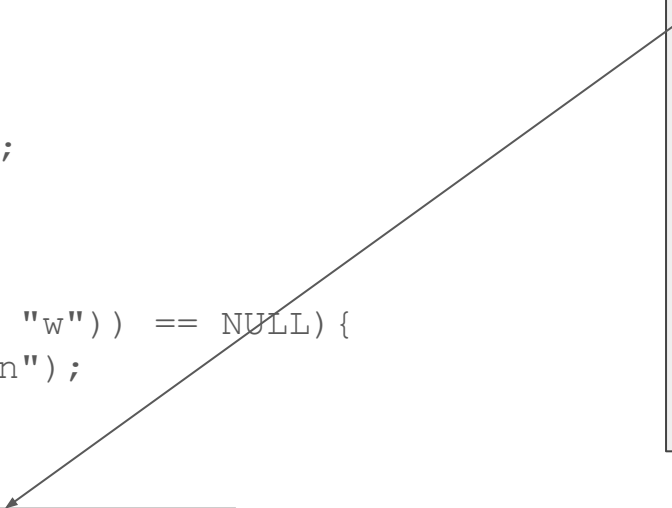
```
#include <stdio.h>
#include <stdlib.h>
void main(void){
    int numbers[] = {1,2,3,4,5};
    int written;

    FILE *f;
    if ((f = fopen("test_file", "w")) == NULL){
        printf("Error opening file\n");
        return;
    }

    written = fwrite((const void*)numbers, sizeof(int), 5, f);
    printf("Wrote %d elements of size %ld\n", written, sizeof(int));

    fclose(f);
}
```

size_t **fwrite**(
const void *buffer,
size_t size,
size_t count,
FILE *stream);



fwrite example with basic types

```
#include <stdio.h>
#include <stdlib.h>
void main(void){
    int numbers[] = {1,2,3,4,5};
    int written;

    FILE *f;
    if ((f = fopen("test_file", "w")) == NULL){
        printf("Error opening file\n");
        return;
    }

    written = fwrite((const void*)numbers, sizeof(int), 5, f);
    printf("Wrote %d elements of size %ld\n", written, sizeof(int));

    fclose(f);
}
```

size_t **fwrite**(

const void *buffer,

size_t size,

size_t count,

FILE *stream);

fwrite example with basic types

```
$ ./fwrite  
Wrote 5 elements of size 4
```

```
$ cat test_file  
00000000  
00000000
```

```
$ xxd test_file  
00000000: 0100 0000 0200 0000 0300 0000 0400 0000  ....  
00000010: 0500 0000 0a00 0000                ....
```

fwrite example with basic types

```
$ ./fwrite  
Wrote 5 elements of size 4
```

```
$ cat test_file  
00000000  
00000000
```

```
$ xxd test_file  
00000000: 0100 0000 0200 0000 0300 0000 0400 0000  . . . . .  
00000010: 0500 0000 0a00 0000  . . . . .
```

fwrite example with complex types

```
typedef struct { char name[10]; int year; int group; } student_t;
void main(void){
    student_t students[] = {"mike", 3, 1}, {"john", 3, 1}, {"andy", 3, 2};;
    int written;

    FILE *f;
    if ((f = fopen("test_file", "w")) == NULL){
        printf("Error opening file\n");
        return;
    }

    written = fwrite((const void*)students, sizeof(student_t), 3, f);
    printf("Wrote %d elements of size %ld\n", written, sizeof(student_t));

    fclose(f);
}
```

fwrite example with complex types

```
$ ./fwrite  
Wrote 3 elements of size 20
```

```
$ cat test_file  
mike   john   andy   vi
```

```
$ xxd test_file  
00000000: 6d69 6b65 0000 0000 0000 f000 0300 0000  mike.....  
00000010: 0100 0000 6a6f 686e 0000 0000 0000 0000  ....john.....  
00000020: 0300 0000 0100 0000 616e 6479 0000 0000  ....andy....  
00000030: 0000 f1cc 0300 0000 0200 0000                .....  
00000040: 0000 0000 0000 0000 0000 0000 0000 0000  .....
```

Low level: fread

The inverse of fwrite. An array of **count** elements of size **size** is read from the file and stored at the memory address **buffer**.

size_t **fread**(void *buffer, size_t size, size_t count, FILE *stream);

- Data is stored at the address **buffer** (no longer const, why?)
- Data is considered to be an array of **count** elements, each of **size** bytes
- All the **count** elements are written to the **stream** file.

File must be opened in “r”, “r+”, or “a+” mode.

Opening in other modes will truncate the file (or not allow reading).

fread example with basic types

```
void main(void){
    int numbers[5];
    int numread;

    FILE *f;
    if ((f = fopen("test_file", "r")) == NULL){
        printf("Error opening file\n"); return;
    }
    numread = fread((void*)numbers, sizeof(int), 5, f);

    printf("Read %d elements of size %ld:", numread, sizeof(int));
    for (;numread>0;numread--) printf("%d ", numbers[5-numread]);
    printf("\n");

    fclose(f);
}
```


fread example with basic types

```
void main(void){
    int numbers[5];
    int numread;

    FILE *f;
    if ((f = fopen("test_file", "r")) == NULL){
        printf("Error opening file\n"); return;
    }
    numread = fread((void*)numbers, sizeof(int), 5, f);

    printf("Read %d elements of size %ld:", numread, sizeof(int));
    for (;numread>0;numread--) printf("%d ", numbers[5-numread]);
    printf("\n");

    fclose(f);
}
```

```
$ ./fread2
Read 5 elements of size 4:1 2 3 4 5
```

fread example with complex types

```
void main(void){
    student_t students[3];
    int numread;

    FILE *f;
    if ((f = fopen("test_file", "r")) == NULL){
        printf("Error opening file\n"); return;
    }

    numread = fread((void*)students, sizeof(student_t), 3, f);
    printf("Read %d elements of size %ld:\n", numread, sizeof(student_t));
    for(;numread>0;numread--)
        printf("%s year %d group %d\n", students[3-numread].name,
            students[3-numread].year, students[3-numread].group);
    fclose(f);
}
```

fread example with complex types

```
void main(void) {
    student_t students[3];
    int numread;

    FILE *f;
    if ((f = fopen("test_file", "r")) == NULL) {
        printf("Error opening file\n"); return;
    }

    numread = fread((void*)students, sizeof(student_t), 3, f);
    printf("Read %d elements of size %ld:\n", numread, sizeof(student_t));
    for(; numread>0; numread--)
        printf("%s year %d group %d\n", students[3-numread].name,
              students[3-numread].year, students[3-numread].group);

    fclose(f);
}
```

```
$ ./fread
Read 3 elements of size 20:
mike year 3 group 1
john year 3 group 1
andy year 3 group 2
```

High level: fprintf

```
int fprintf(FILE *f, const char *format_string, ...);
```

Works exactly like **printf** only the output is into the file **f** (that must be opened).

In fact, printf is equivalent to

```
fprintf(stdout, ..., ...);
```

High level: fprintf example

```
void main(void){
    student_t students[] = { {"mike", 3, 1}, {"john", 3, 1}, {"andy", 3,
2}};
    int i;

    FILE *f;
    if ((f = fopen("test_file", "w")) == NULL){
        printf("Error opening file\n");
        return;
    }

    for (i=0;i<3;i++)
        fprintf(f, "%s %d %d\n",
                students[i].name, students[i].year, students[i].group);
    fprintf(stdout, "Done!\n");
    fclose(f);
}
```

High level: fprintf example

```
void main(void){
    student_t students[] = { {"mike", 3, 1}, {"john", 3, 1}, {"andy", 3,
2}};
    int i;

    FILE *f;
    if ((f = fopen("test_file", "w")) == NULL){
        printf("Error opening file\n");
        return;
    }

    for (i=0;i<3;i++)
        fprintf(f, "%s %d %d\n",
                students[i].name, students[i].year, students[i].group);
    fprintf(stdout, "Done!\n");
    fclose(f);
}
```

```
$ cat test_file
mike 3 1
john 3 1
andy 3 2
```

```
$ ./fwrite_highlevel
Done!
```

High level: fscanf

```
int fscanf(FILE *f, const char *format_string, ...);
```

Works exactly like **scanf** only the input is **from** the file **f** (that must be opened).

In fact, scanf is equivalent to

```
fscanf(stdin, ..., ...);
```

File input output

Low level operations: **fread**, **fwrite**

- Useful for writing bytes
- Useful for writing complex types (more concise)
- Results in **binary** files (cannot be printed out)

High level operations: **fprintf**, **fscanf**

- Works with text, printable characters
- Results in **text** files (can be printed)

More

```
#include <stdio.h>
```

```
int fgetc(FILE *stream); // read a single character (like getchar() on a file)
```

```
char *fgets(char *s, int size, FILE *stream); // read a string of size characters
```

```
int getc(FILE *stream); // same as fgetc
```

```
int ungetc(int c, FILE *stream); // puts the char c back in the file input buffer
```

Other file operations

Flushing the input/output buffers: `int fflush(FILE *f);`

- Writes any bytes remaining in the input buffer
- Discards bytes unread in the output buffer

Seeking in the file (moving the file position):

SEEK_SET: start
SEEK_CUR: current
SEEK_END: end

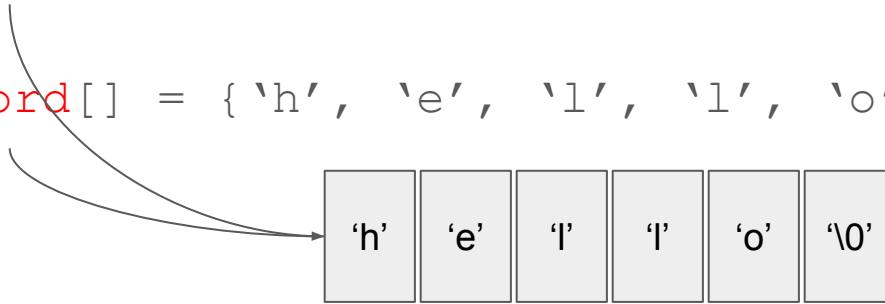
- `int fseek(FILE *stream, long offset, int whence);`
- Changes position in the file by **offset** from the **whence** position
- Useful in binary files for jumping to specific records

End of file: `int feof(FILE *f);` returns non-zero (true) if end of file reached.

Strings

Internal representation

```
char word[] = "hello";  
// or  
char word[] = { 'h', 'e', 'l', 'l', 'o', '\\0' };
```



Char array, end with the null character '\\0' (which is really 0).

'\\0' acts as the string terminator.

String length

How can we determine the length of a string?

- Think about the string terminator ('\0')

String length

How can we determine the length of a string?

- Think about the string terminator ('\0')

```
int length(const char *string){  
    int i=0;  
    while (string[i] != '\0') i++;  
    return i;  
}
```

String length

How can we determine the length of a string?

- Think about the string terminator ('\0')

```
int length(const char *string){  
    int i=0;  
    while (string[i] != '\0') i++;  
    return i; // Return length + 1  
}
```

String length

How can we determine the length of a string?

- Think about the string terminator ('\0')

```
int length(const char *string){  
    int i=0;  
    while (string[i] != '\0') i++;  
    return i-1;    // Return length  
}
```


String length

How can we determine the length of a string?

- Think about the string terminator ('\0')

```
int length(const char *string){  
    int i=0;  
    while (string[i] != '\0') i++;  
    return i-1;    // For empty strings returns -1  
}
```

String length

How can we determine the length of a string?

- Think about the string terminator ('\0')

```
int length(const char *string){  
    int i=0;  
    while (string[i++] != '\0');  
    return i-1;  
}
```

String length

How can we determine the length of a string?

- Think about the string terminator ('\0')

```
int length(const char *string){  
    int i=0;  
    while (string[i++] != '\0');  
    return i-1;  
}
```

A similar function is defined in <string.h>, int **strlen**(const char *s);

String length

```
char name[50];  
char *names[10];  
int i;  
for (i=0; i<10; i++){  
    scanf("%s", name);  
    names[i] = malloc(strlen(name)+1);  
    ...  
}
```

Copying strings

Let's implement:

```
void copy(const char *source, char *dest){  
    while (source[i] != '\0'){ // like in strlen  
        dest[i] = source[i];  
        i++;  
    }  
    dest[i] = source[i];  
}
```

Copying strings

Let's implement:

```
void copy(const char *source, char *dest){  
    while (source[i] != '\\0'){ // like in strlen  
        dest[i] = source[i];  
        i++;  
    }  
    dest[i] = source[i]; // this copies the '\\0'  
}
```

Copying strings

Let's implement:

```
void copy(const char *source, char *dest){  
    while (source[i] != '\\0'){ // like in strlen  
        dest[i] = source[i++];  
    }  
    dest[i] = source[i]; // this copies the '\\0'  
}
```

Copying strings

Let's implement:

```
void copy(const char *source, char *dest){  
    while ((dest[i] = source[i++]) != '\0');  
    dest[i-1] = '\0';    // terminate dest string  
}
```


Copying strings

Let's implement:

```
void copy(char *dest, const char *source){  
    while ((dest[i] = source[i++]) != '\\0');  
    dest[i-1] = '\\0';    // terminate dest string  
}
```

`<string.h>` provides the functions

```
char *strcpy(char *dest, const char *src);  
char *strncpy(char *dest, const char *src, size_t n); // safer!
```

Copying strings: example

```
char name[50];  
char *names[10];  
int i;  
for (i=0; i<10; i++){  
    scanf("%s", name);  
    names[i] = malloc(strlen(name)+1);  
    strcpy(names[i], name);  
    ...  
}
```

Concatenating strings

In Python:

```
S1 = 'hello '  
S2 = 'world'  
print(S1+S2)
```

Concatenating strings

In Python:

```
S1 = 'hello '  
S2 = 'world'  
print(S1+S2)
```

In C:

```
#include <string.h>  
  
char S1[] = "hello ";  
  
char S2[] = "world";  
  
printf("%s\n", S1+S2);
```

Concatenating strings

In Python:

```
S1 = 'hello '  
S2 = 'world'  
print(S1+S2)
```

S1 is the address of the start of the S1 array.
S2 is the address of the start of the S2 array.
S1 + S2 is another memory address. If we try to read it we probably
get a Segmentation Fault

In C:

```
#include <string.h>  
  
char S1[] = "hello ";  
  
char S2[] = "world";  
  
printf("%s\n", S1+S2)
```



Concatenating strings

In Python:

```
S1 = 'hello '  
S2 = 'world'  
print(S1+S2)
```

In C:

```
#include <string.h>  
char S1[] = "hello ";  
char S2[] = "world";  
strcat(S1, S2);  
  
printf("%s\n", S1);
```

In C:

```
char *strcat(char *dest, const char *src);  
char *strncat(char *dest, const char *src, size_t n);
```

Concatenating strings

In Python:

```
S1 = 'hello '  
S2 = 'world'  
print(S1+S2)
```

In C:

```
#include <string.h>  
char S1[] = "hello ";  
char S2[] = "world";  
strcat(S1, S2);  
  
printf("%s\n", S1);
```

In C:

```
char *strcat(char *dest, const char *src);  
char *strncat(char *dest, const char *src, size_t n);
```



Concatenating strings

In Python:

```
S1 = 'hello '  
S2 = 'world'  
print(S1+S2)
```

In C:

```
#include <string.h>  
char S1[] = "hello ";  
char S2[] = "world";  
strcat(S1, S2);  
  
printf("%s\n", S1);
```

In C:

```
char *strcat(char *dest, const char *src);  
char *strncat(char *dest, const char *src, size_t n);
```

dest must be large enough to accommodate the extra characters in src



Concatenating strings

In Python:

```
S1 = 'hello '  
S2 = 'world'  
print(S1+S2)
```

In C:

```
#include <string.h>  
char dest[50];  
Char S1[] = "hello ";  
char S2[] = "world";  
strcpy(dest, S1);  
strcat(dest, S2);  
printf("%s\n", S1);
```

In C:

```
char *strcat(char *dest, const char *src);  
char *strncat(char *dest, const char *src, size_t n);
```

dest must be large enough to accommodate the extra characters in src

Comparing strings

In Python:

```
S1 = input()
S2 = input()
if s1 == s2:
    print('Match')
```

In C:

```
char *S1 = malloc(10);
char *S2 = malloc(10);
scanf("%s %s", S1, S2);
if (S1 == S2) {
    printf("Match!\n");
}
```

What does the code do?

Comparing strings

In Python:

```
S1 = input()
S2 = input()
if s1 == s2:
    print('Match')
```

S1 is the address of the start of the S1 array.

S2 is the address of the start of the S2 array.

We are comparing to memory addresses. Because they are both allocated with malloc they will always be different (unless NULL).

In C:

```
char *S1 = malloc(10);
char *S2 = malloc(10);
scanf("%s %s", S1, S2);
if (S1 == S2) {
    printf("Match!\n");
}
```

What does the code do?

Comparing strings in C

```
#include <string.h>
int strcmp(const char *s1, const char *s2);
int strncmp(const char *s1, const char *s2, size_t n);
```

How they work:

- Compare pairs of characters ($s1[i]$, $s2[i]$) until they are different (or until the end of the strings)
- At the first pair of characters ($s1[i]$, $s2[i]$) that is different return $s1[i] - s2[i]$

Returns: 0 if $s1 == s2$; >0 if $s1 > s2$; <0 if $s1 < s2$.

Comparing strings

In Python:

```
S1 = input()
S2 = input()
if s1 == s2:
    print('Match')
```

In C:

```
char *S1 = malloc(10);
char *S2 = malloc(10);
scanf("%s %s", S1, S2);
if (strcmp(S1, S2) == 0) {
    printf("Match!\n");
}
```

Writing into strings and Reading from strings

```
int sprintf(char *s, const char *format, ...);
```

```
struct student students[3] = {"mike", 3, 1}, {"john", 3, 1}, {"andy", 3, 2};
char *json_rep = malloc(100);
int pos = 0;
int i=0;
json_rep[pos++] = '{'; json_rep[pos++] = '[';
for (i=0; i<3; i++){
    int numchars;
    numchars = sprintf(&json_rep[pos], "{name=%s, year=%d, group=%d}, \n",
                      students[i].name, students[i].year, students[i].group);
    pos += numchars;
}
json_rep[pos++] = ']';
json_rep[pos++] = '\0';
printf("JSON REP: \n%s\n", json_rep);
```

```
$ ./sprintf
JSON REP:
[{name=mike, year=3, group=1},
{name=john, year=3, group=1},
{name=andy, year=3, group=2},
]
```

Writing into strings and Reading from strings

```
int sprintf(char *s, const char *format, ...);
```

sprintf always writes from the start of the string.

However, the string is just an address in memory. We can advance it as we write.

```
for (i=0;i<3;i++){  
    int numchars;  
    numchars = sprintf(&json_rep[pos], "{name=%s,year=%d,group=%d},\n",  
                      students[i].name, students[i].year, students[i].group);  
    pos += numchars;
```

```
$ ./sprintf  
JSON REP:  
[{name=mike,year=3,group=1},  
{name=john,year=3,group=1},  
{name=andy,year=3,group=2},  
]
```

Writing into strings and Reading from strings

```
for (i=0;i<3;i++){  
    int numchars;  
    numchars = sprintf(&json_rep[pos], "{name=%s,year=%d,group=%d},\n",  
                      students[i].name, students[i].year, students[i].group);  
    pos += numchars;  
}
```

pos=2 ▼

i=0

{ [

```
$ ./sprintf  
JSON REP:  
[{name=mike,year=3,group=1},  
{name=john,year=3,group=1},  
{name=andy,year=3,group=2},  
]
```


Writing into strings and Reading from strings

```
for (i=0;i<3;i++){  
    int numchars;  
    numchars = sprintf(&json_rep[pos], "{name=%s,year=%d,group=%d},\n",  
                      students[i].name, students[i].year, students[i].group);  
    pos += numchars;  
}
```

pos=2 ▼

i=0

{ [{name=mike, year=3, group=1} ,

numchars

```
$ ./sprintf  
JSON REP:  
[{name=mike,year=3,group=1},  
{name=john,year=3,group=1},  
{name=andy,year=3,group=2},  
]
```

Writing into strings and Reading from strings

```
for (i=0;i<3;i++){  
    int numchars;  
    numchars = sprintf(&json_rep[pos], "{name=%s,year=%d,group=%d},\n",  
                      students[i].name, students[i].year, students[i].group);  
    pos += numchars;  
}
```

pos=2+numchars ▼

i=1

{ [{name=mike, year=3, group=1} ,

numchars

```
$ ./sprintf  
JSON REP:  
[{name=mike,year=3,group=1},  
{name=john,year=3,group=1},  
{name=andy,year=3,group=2},  
]
```

Writing into strings and Reading from strings

```
for (i=0;i<3;i++){  
    int numchars;  
    numchars = sprintf(&json_rep[pos], "{name=%s,year=%d,group=%d},\n",  
                      students[i].name, students[i].year, students[i].group);  
    pos += numchars;
```

pos=2+numchars ▼

i=1

{ [{name=mike, year=3, group=1} , {n...

numchars

```
$ ./sprintf  
JSON REP:  
[{name=mike,year=3,group=1},  
{name=john,year=3,group=1},  
{name=andy,year=3,group=2},  
]
```

Reading from strings

```
int sscanf(char *s, const char *fmt, args...);
```

Other functionality

Many more things in `string.h`:

- Searching in strings: `strchr`, `strstr`
- String tokenizer: `strtok`

Working with memory

Allocating/Releasing memory

Memory functions are defined in `stdlib.h` and `string.h`

```
void *malloc(size_t size);
```

- Allocate memory of **size** bytes. Returns memory address or NULL on error.

```
void free(void *ptr);
```

- Release previously allocated memory at address **ptr**

```
void *calloc(size_t nmemb, size_t size);
```

- Allocate array of **nmemb** elements, each **size** bytes. Initialise with zeros.

```
void *realloc(void *ptr, size_t size);
```

- Extend memory allocated at **ptr** by an additional **size** bytes. Return address.

Setting memory contents

```
void *memset(void *ptr, int byte, size_t count);
```

Set the contents of the first **count** bytes at memory address **ptr** with the byte value **byte**.

`memset` is commonly used to clear memory or arrays.

Example:

```
char array[10];  
for (i=0;i<10;i++) printf("%x ", array[i]);  
memset(array, 0, 10);  
for (i=0;i<10;i++) printf("%x ", array[i]);
```

```
$ ./memset  
ffffffb2 fffffffd9 ffffffffe 7f 0 0 0 0 0 0  
0 0 0 0 0 0 0 0 0 0
```


Copying memory

```
void *memcpy(void *dest, const void *src, size_t n);
```

Copies from the memory address **src** **n** bytes into the memory address **dest**.

Similar in functionality with **strncpy**, however that stops at the first '\0'.

Other

Random numbers

```
#include <stdlib.h>
```

```
long int random(void);
```

- Generates a positive random integer
- The number can be converted to a smaller range using the modulo operator:
`random() % MAX; // will return a random number between 0 and MAX`

```
void srand(unsigned int seed);
```

- Initialises the pseudo-random number generator
- Using the same **seed** twice will result in the same sequence of random numbers.

Time

```
#include <time.h>
```

```
time_t time(time_t *tloc);
```

Returns the number of seconds since the “Epoch” (01-01-1970 00:00:00).

If **tloc** is not NULL **time** will also write the returned value at that address.

Example:

```
printf("Seconds elapsed = %ld\n", time(NULL));
```

```
> 1582753506
```

- `time` is useful as a seed for `srandom`

Command line arguments

```
> ./test arg1 arg2 arg3...
```

In Java

```
public static void main(String args[]) {  
  
    if (args.size() < X) {  
  
        System.out.println("Must have X arguments");  
  
        System.exit(1);  
  
    }  
  
    System.out.println(args[0]);  
  
    System.out.println(args[1]);  
}
```

In Python

```
import sys

print("Number of arguments is %d" % len(sys.argv))

for a in sys.argv:

    print("Argument: %s" % a)
```


arg0



```
> ./test arg1 arg2 arg3...
```

Command line arguments are passed to the program as a list of strings.

The first argument is always the name of the executable.

In C

Number of arguments

Argument list



```
void main(int argc, char *argv[]) {  
  
    if (argc < X) {  
  
        printf("Must have X arguments\n");  
  
        return;  
  
    }  
  
    printf("%s\n", argv[0]);  
  
    printf("%s\n", argv[1]);  
  
}
```

Example

```
void main(int argc, char*argv[]){  
    if (argc < 2){  
        printf("Usage: %s <name>\n", argv[0]);  
        return;  
    }  
  
    printf("Hello %s\n", argv[1]);  
}
```

